



NOAA/EMC Environmental Modeling Center

Global Workflow Development Team

January 2026

- 1 The GitLab CI/CD Paradigm
- 2 Pipeline Stages
- 3 Template System
- 4 Multi-Modality: PR Cases & CTests
- 5 Host-Specific Extensibility
- 6 Conditional Execution
- 7 Error Handling & Reporting
- 8 Summary

Declarative (GitLab YAML)

- Specify **what** to do, not **how**
- Configuration as code
- Self-documenting pipelines
- Reproducible builds
- Version controlled alongside code

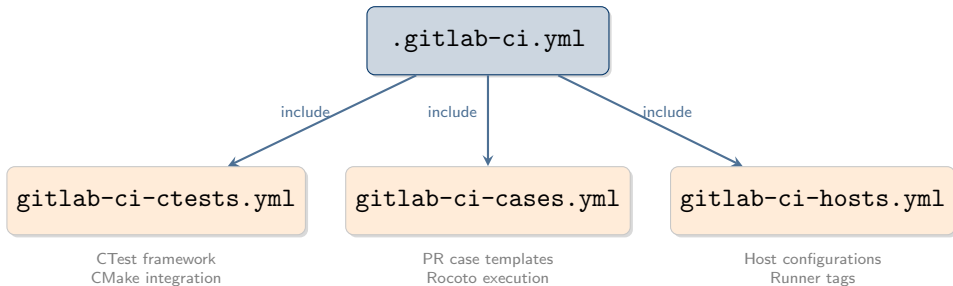
```
build-hera:  
  stage: build  
  tags: [hera]  
  script:  
    - make build
```

Imperative (Traditional Scripts)

- Step-by-step instructions
- External script files
- Complex conditionals
- Environment-dependent
- Harder to maintain

```
#!/bin/bash  
if [ "$MACHINE" = "hera" ]; then  
  ssh hera "cd $DIR && make"  
fi
```

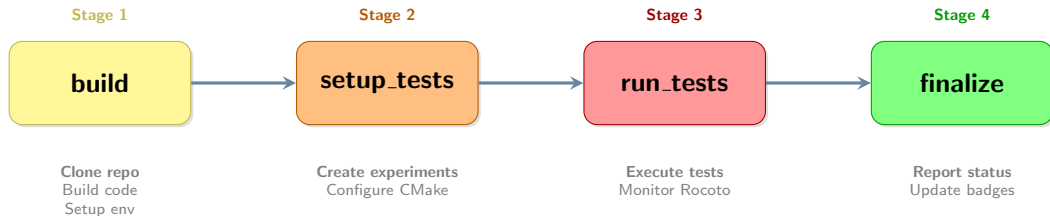
Key Insight: GitLab's YAML declaratively defines *what* should happen; the GitLab Runner determines *how* to execute it on each platform.



.gitlab-ci.yml – Include Directive

```
include:  
- local: 'dev/ci/gitlab-ci-ctests.yml'  
- local: 'dev/ci/gitlab-ci-cases.yml'  
- local: 'dev/ci/gitlab-ci-hosts.yml'
```

Modular Design: Split configuration by concern for maintainability



Stage Definition

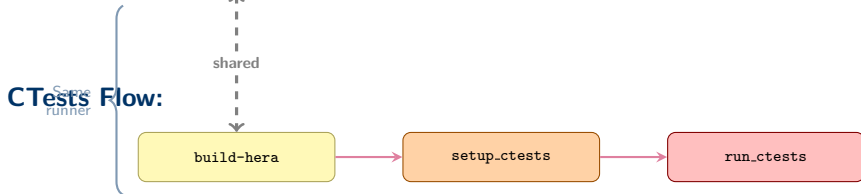
```
stages:  
- build  
- setup_tests  
- run_tests  
- finalize
```

Sequential Execution: Jobs in stage N must complete before stage N+1 begins
Parallel Within Stage: Multiple jobs in same stage run concurrently

PR Cases Flow:



CTests Flow:



PIPELINE_TYPE Variable Controls Flow

```
variables:
  PIPELINE_TYPE: pr_cases

rules:
  - if: $PIPELINE_TYPE == "pr_cases" && $CI_PIPELINE_SOURCE == "trigger"
  - if: $PIPELINE_TYPE == "ctests" && $CI_PIPELINE_SOURCE == "trigger"
```

YAML Anchors (&)

Define reusable data blocks

```
.hera_cases_matrix: &hera_cases
- caseName:
  - "C48_ATM"
  - "C48_S2SW"
  - "C96_atm3DVar"

setup_experiments-hera:
  parallel:
    matrix: *hera_cases
```

Anchors copy data structures verbatim
Use &name to define, *name to reference

extends Keyword

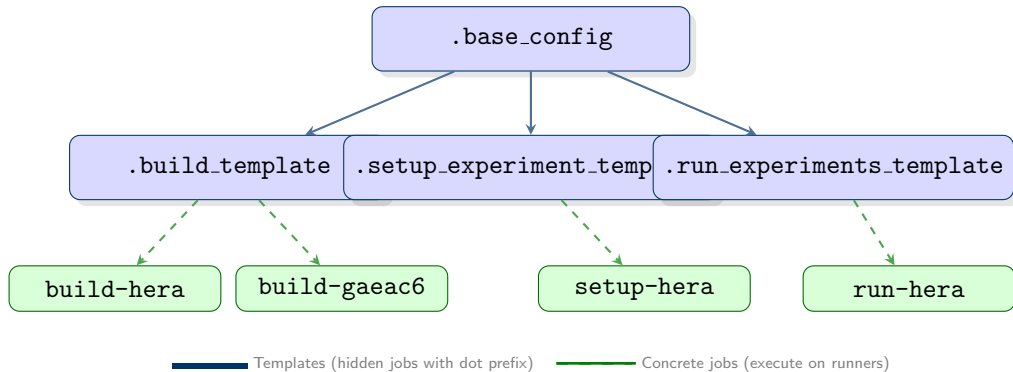
Inherit and override job properties

```
.build_template:
  stage: build
  script:
    - git submodule update
    - dev/ci/scripts/ci_utils.sh build

build-hera:
  extends: .build_template
  variables:
    machine: hera
  tags: [hera]
```

extends merges configs and allows overrides

DRY Principle: Define once in template, instantiate per-host



Multiple Inheritance with extends

```
.build_template:
  extends:
    - .base_config
    - .failure_cleanup_template
  stage: build
  script:
```


.base_config – Shared by ALL Jobs

```
.base_config:
  variables:
    badge_GIST_ID: "66059582886cb5c2485ff64bf24e7f93"
    GIT_STRATEGY: none
  before_script:
    - |
      echo "Starting GitLab CI pipeline for global-workflow project"
      PR_NUMBER=${PR_NUMBER:-0}

      export GW_RUN_PATH="${CI_BUILDS_DIR}/${WORKSPACE_ID}"
      export GW_HOMEgfs="${GIT_CLONE_PATH}"
      export RUNTESTS_DIR="${GW_RUN_PATH}/RUNTESTS"

      export GH=$(which gh 2>/dev/null || echo "${HOME}/bin/gh")

      if [ ! -d "${GW_HOMEgfs}" ]; then
        echo "ERROR: GW_HOMEgfs directory does not exist"
        exit 1
      fi
```

before_script runs before every job's main script
Sets up environment variables, validates paths, configures GitHub CLI

PR Cases

- End-to-end workflow tests
- Full Rocoto execution
- Multiple forecast cycles
- GFS, GEFS, S2S, GCAFS configs
- ~15 cases per host
- Runtime: 2–8 hours

Triggered by: PR, Schedule

VS

CTests (CMake)

- Functional unit tests
- Single job validation
- Quick feedback (~30 min)
- Uses `ctest -L <label>`
- JUnit XML reports
- GitHub API triggered

Triggered by: GitHub API

PIPELINE_TYPE Variable Switches Modality

```
variables:  
  PIPELINE_TYPE: pr_cases
```

gitlab-ci-ctests.yml – CTest Templates

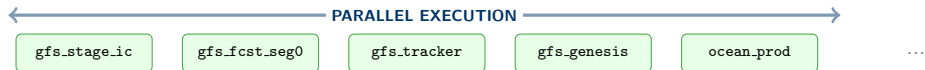
```
.setup_ctests_template:
  extends:
    - .base_config
    - .failure_cleanup_template
  stage: setup_tests
  script: |
    source "${GW_HOMEgfs}/dev/ush/gw_setup.sh"
    cd "${GW_HOMEgfs}/dev/ctests"
    mkdir -p build && cd build
    cmake -S "${GW_HOMEgfs}"
    ctest -N

.run_ctests_template:
  stage: run_tests
  script: |
    cd "${GW_HOMEgfs}/dev/ctests/build"
    ctest -L "${CTEST_NAME}" --output-junit result.xml --stop-on-failure
  artifacts:
    reports:
      junit: ${GIT_CLONE_PATH}/dev/ctests/build/result.xml
```

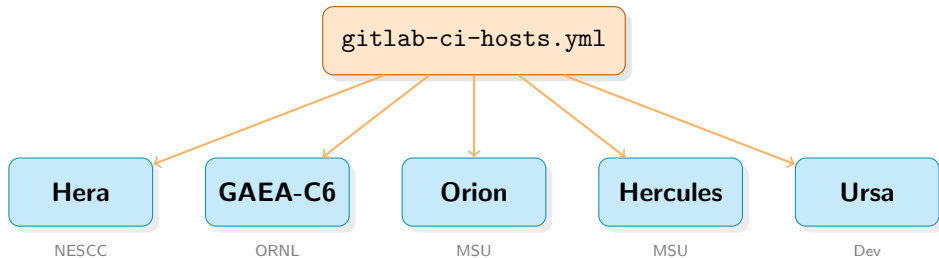
CTests use CMake's testing framework with JUnit XML output
GitLab automatically parses JUnit for test result visualization

CTest Label Matrix – Each Runs as Separate Job

```
.ctests_cases_template:  
  extends: .run_ctests_template  
  parallel:  
    matrix:  
      - CTEST_NAME:  
        - C48_ATM-gfs_stage_ic  
        - C48_ATM-gfs_fcst_seg0  
        - C48_ATM-gfs_atmos_prod_f000-f002  
        - C48_ATM-gfs_tracker  
        - C48_ATM-gfs_genesis  
        - C48_S2SW-gfs_fcst_seg0  
        - C48_S2SW-gfs_ocean_prod_f006  
        - C48_S2SW-gfs_ice_prod_f006  
        - C48_S2SWA-gefs-gefs_fcst_mem001_seg0
```



9 CTests run in parallel across available runners



Host-Specific Case Matrices (YAML Anchors)

```
.hera_cases_matrix: &hera_cases
- caseName: ["C48_ATM", "C48_S2SW", "C48_S2SWA_gefs", "C48mx500_3DVarA0WCDA",
  "C96C48_hybatmDA", "C96C48_hybatmsnowDA", "C96_atm3DVar", ...]

.gaeac6_cases_matrix: &gaeac6_cases
- caseName: ["C48_ATM", "C48_S2SW", "C96_atm3DVar", ...]

.orion_cases_matrix: &orion_cases
- caseName: ["C48_ATM", "C48_S2SW", "C96_atm3DVar", ...]
```

Each HPC platform runs a **different subset of tests**

Concrete Jobs Inherit Templates + Host-Specific Config

```
setup_experiments-hera:
  extends: .setup_experiment_template
  variables:
    machine: hera
  tags:
    - hera
  parallel:
    matrix: *hera_cases
  needs:
    - build-hera
  rules:
    - if: $PIPELINE_TYPE == "pr_cases" &&
        ($CI_PIPELINE_SOURCE == "trigger" || $CI_PIPELINE_SOURCE == "schedule") &&
        ($RUN_ON_MACHINES =~ /\bhera\b|all/)

run_experiments-hera:
  extends: .run_experiments_template
  variables:
    machine: hera
  tags: [hera]
  parallel:
    matrix: *hera_cases
  needs: [setup_experiments-hera]
```

To add a new HPC platform (e.g., “jet”):

Step 1: Define case matrix

```
.jet_cases_matrix: &jet_cases
- caseName:
  - "C48_ATM"
  - "C48_S2SW"
```

Step 2: Add build job

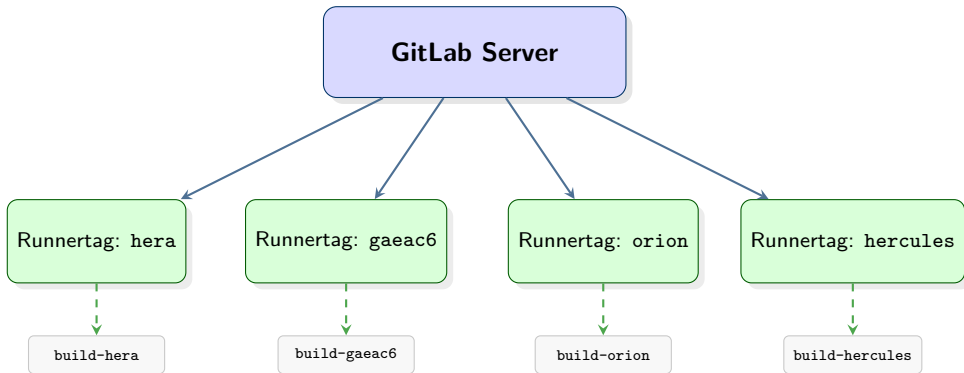
```
build-jet:
  extends: .build_template
  variables:
    machine: jet
  tags: [jet]
  rules:
    - if: $RUN_ON_MACHINES =~ /jet|all/
```

Step 3: Add setup/run jobs

```
setup_experiments-jet:
  extends: .setup_experiment_template
  variables:
    machine: jet
  tags: [jet]
  parallel:
    matrix: *jet_cases
  needs: [build-jet]
```

```
run_experiments-jet:
  extends: .run_experiments_template
  variables:
    machine: jet
  tags: [jet]
  parallel:
    matrix: *jet_cases
  needs: [setup_experiments-jet]
```

Only need to update host file by following pattern



Jobs route to runners based on tags matching

Tags Route Jobs to Correct Runner

```
build-hera:
  tags:
    - hera
```


Rule Example

```
rules:  
  - if: $PIPELINE_TYPE == "pr_cases" &&  
        ($CI_PIPELINE_SOURCE == "trigger" ||  
         $CI_PIPELINE_SOURCE == "schedule") &&  
        ($RUN_ON_MACHINES =~ /\bhera\b|all/)
```

AND

PIPELINE_TYPE?

pr_cases — ctests

AND

CI_PIPELINE_SOURCE?

trigger — schedule — push

RUN_ON_MACHINES?

hera — gaeac6 — all

Rules prevent accidental runs: Jobs only execute when triggered correctly, not on merge to develop or manual push

.failure_cleanup_template – after_script

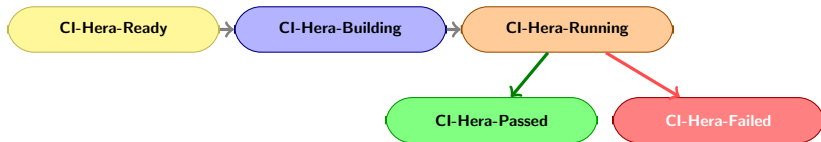
```
.failure_cleanup_template:
  after_script:
    - |
      if [[ ${CI_JOB_STATUS} == "failed" ]]; then
        Machine="${machine~}"

        if [[ "${GFS_CI_RUN_TYPE}" == "pr_cases" && "${PR_NUMBER}" != 0 ]]; then
          ${GH} pr edit ${PR_NUMBER} --repo ${GW_REPO_URL} \
            --add-label "CI-${Machine}-Failed" \
            --remove-label "CI-${Machine}-Building" \
            --remove-label "CI-${Machine}-Running"

          comment_body="**BUILD FAILED** on ${Machine}..."
          ${GH} pr comment ${PR_NUMBER} --body "${comment_body}"

        elif [[ "${GFS_CI_RUN_TYPE}" == "nightly" ]]; then
          curl "https://img.shields.io/badge/${machine}_nightly-failed-red" \
            -o badge.svg
          ${GH} gist edit "${badge_GIST_ID}" --add badge.svg
        fi
      fi
```

PR Label Lifecycle (updated via GitHub CLI):



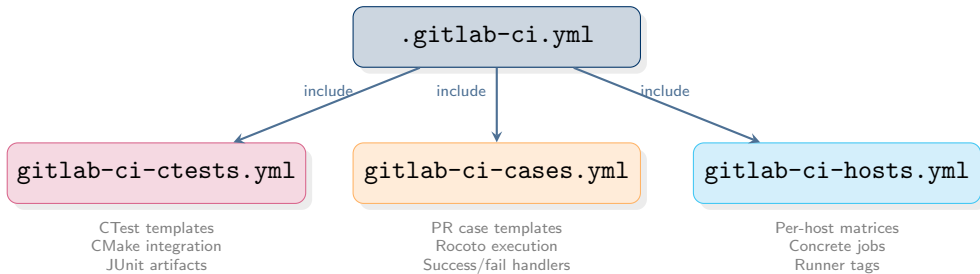
Nightly Badges (GitHub Gist):

hera_nightly — passed

gaeac6_nightly — failed

Bidirectional Integration:

GitHub triggers GitLab → GitLab updates GitHub labels/comments/badges



Key Design Principles

- Declarative YAML configuration
- DRY via templates and anchors
- Separation of concerns
- Easy host extensibility

Benefits Achieved

- 5 HPC platforms supported
- 2 testing modalities (PR/CTest)
- Automated GitHub integration
- ~50 lines to add new host

cc

Construct	Syntax	Purpose
Hidden job	.template_name:	Define reusable template (not executed)
Extends	extends: .template	Inherit from template(s)
Anchor	&name	Define reusable data block
Alias	*name	Reference anchor data
Include	include: - local:	Import from other YAML files
Parallel matrix	parallel: matrix:	Generate multiple jobs from list
Tags	tags: [host]	Route job to specific runner
Rules	rules: - if:	Conditional job execution
Needs	needs: [job]	Explicit job dependency
Artifacts	artifacts: reports:	Collect test results



Thank you!

GitHub: `github.com/NOAA-EMC/global-workflow`

GitLab: `GitLab vLab (internal)`

CI Docs: `dev/ci/ directory`

Global Workflow CI/CD Architecture
NOAA/EMC Environmental Modeling Center